CentraleSupélec

2EL1520 – Génie logiciel orienté objet

Supermarket Checkout System

Object-Oriented Software Engineering Project

Author:

Enzo Jardim Vendramin

Teacher: Paolo Ballarini

Academic Year: 2025–2026

31 May 2026

Contents

1 Introduction **2**
- 1.1 Context and Objectives . . . 2
- 1.2 Scope and Deliverables . . . 2
- 1.3 Report Structure . . . 2

I The Specified System **3**

2 Requirements and Design Decisions **3**
- 2.1 Functional Requirements and Their Realisation . . . 3
- 2.2 The Bill Computation Pipeline . . . 4

3 Architecture, Design Patterns and UML Models **5**
- 3.1 Package Architecture . . . 5
- 3.2 Design Patterns . . . 5
- 3.3 Domain Core and Orchestration . . . 6
- 3.4 Class Diagram (per package) . . . 6
- 3.5 Use-Case Diagram . . . 9
- 3.6 Sequence Diagrams . . . 10

4 Testing **13**
- 4.1 JUnit Test Strategy . . . 13
- 4.2 Test Scenarios . . . 13
- 4.3 How to Reproduce the Tests . . . 13

II Extensions Beyond the Specification **15**

5 The Optional Extensions **15**
- 5.1 Undo/Redo (Command Pattern) . . . 15
- 5.2 Promotions Engine (Strategy) . . . 16
- 5.3 Per-Category VAT . . . 16
- 5.4 Loyalty Points . . . 16
- 5.5 The Extended Bill and a Demonstration . . . 16
- 5.6 JavaFX Graphical Interface . . . 16

6 Design Analysis and Conclusion **18**
- 6.1 Advantages of the Solution . . . 18
- 6.2 Workload Split . . . 18
- 6.3 Conclusion . . . 19

A How to Build and Run **20**

B Complete Class Diagram **20**

C Selected Code Listings **22**

Chapter 1

Introduction

This report documents the design and implementation of a *supermarket checkout system* developed in Java for the course *2EL1520 — Génie logiciel orienté objet*. The system processes the items a customer buys at a checkout, computes the corresponding bill, handles the bank-card payment, and keeps the shop's inventory up to date in real time.

1.1 Context and Objectives

A supermarket checkout system is a natural setting for object-oriented design: it brings together several collaborating components — the *cash register* (which records the bought items and computes the total bill), the *customer* (who may adhere to a discount plan and request home delivery), and the *bank payment system* (which authorises the card transaction). Each component has clear responsibilities and well-defined interactions.

The objective of this work is therefore twofold: to implement the full set of functional requirements, and to do so with a clean, pattern-driven design that remains open to extension. This report emphasises *why* the code is structured the way it is, and shows explicitly that every required feature has been implemented and tested.

1.2 Scope and Deliverables

The deliverables associated with this project are:

- the Java implementation of the supermarket checkout system, built with Maven and tested with JUnit 5;
- an initial configuration file (`my_supermarket.ini`) and end-to-end test scenarios executed through the command-line interface;
- the UML models (class, use-case and sequence diagrams);
- this report.

The source code is hosted at `https://github.com/enzovendramin/supermarket-checkout`. The repository offers two runnable versions: a *specification version* (tag `v1.0`), containing exactly what the instructions require, and a *final version* (the `master` branch), which adds the optional extensions described in Part II.

1.3 Report Structure

The report is organised in two parts. **Part I** covers the *specified system*: it recalls the requirements and the design decisions they induced (Chapter 2), presents the architecture, the design patterns and the UML models (Chapter 3) and describes the testing (Chapter 4). **Part II** is dedicated to the optional extensions built on top of the specified baseline. The report ends with a design analysis and conclusion, followed by appendices containing the complete class diagram and the build instructions.

Part I

The Specified System

Chapter 2

Requirements and Design Decisions

This chapter recalls the functional requirements, shows that each of them is implemented and tested, and explains the main design decisions they induced.

2.1 Functional Requirements and Their Realisation

The core requirements (R1–R10) describe a checkout that treats purchases as a set of bought items in mandatory categories (R1, R2, R2b), where each item is priced and the price may depend on the quantity (R3), and payment is made by bank card (R4). The system must offer *extensible* discount plans — at least *prime* (20% off when the total reaches €50) and *platinum* (30% off always) — and let the owner introduce new ones (R5, R5b). It must apply *category-level pricing policies* that are likewise extensible (R6, R6b). Customers may request home delivery (R7), charged according to weight and distance, with refusal above 50 kg (R8) and plan-based reductions (R8b). Finally, the system maintains a live inventory that raises low-stock alerts using the Observer pattern (R9) and supports two-hour delivery slots with anti-overbooking and dynamic pricing (R10). Part 2 of the assignment adds a command-line user interface (CLUI) and mandatory tests.

Table 2.1 shows that every element required by the instructions is implemented and tested, mapping each requirement to the classes that realise it and to the JUnit tests (or scenario) that validate it.

Req.	Implemented in	Validated by
R1, R2, R2b	`Cart`, `CartEntry`, `Item`, `Category`; `Supermarket.setup`	`CliDispatcherTest`, `BillComputationTest`
R3	`Item.unitPriceFor` (quantity-based price)	`ItemTest`
R4	`POSDevice`, `TransactionAuthorisationSystem`, `BankCard`	`PaymentFlowTest`, `TransactionAuthorisationSystemTest`
R5	`DiscountPlan` + Normal/Prime/Platinum	`DiscountPlanTest`, `BillComputationTest`
R5b	`DiscountPlanFactory`, `getAnnualFee`	`DiscountPlanTest`, `SupermarketTest`
R6, R6b	`PricingPolicy`, `CategoryDiscount`	`PricingPolicyTest`
R7	`DeliveryRequest`, `Supermarket.requestDelivery`	`DeliveryCheckoutTest`
R8	`StandardDeliveryCalculator`	`DeliveryCalculatorTest`
R8b	`DiscountPlan.applyDeliveryDiscount`	`DiscountPlanTest`, `DeliveryCheckoutTest`
R9	`Inventory`, `StockObserver` (Observer)	`InventoryObserverTest`
R10	`TimeSlot`, `DeliveryManager`	`DeliveryManagerTest`, `TimeSlotTest`
Customer ID (§2.3)	`Customer.numericalId`	`SupermarketTest`
CLUI (Part 2)	`cli` package (login, setup, scanItem, pay, runTest, ...)	`CliDispatcherTest`
Test scenario (§4)	`testScenario1.txt`, `my_supermarket.ini`	`CliDispatcherTest`

Table 2.1: Requirements traceability: every required feature is implemented and tested.

Two words in the instructions drove most of the design: *extensible* and *Observer*. The requirement to add new discount plans (R5b) and new pricing policies (R6b) without modifying existing code led directly to the **Strategy** pattern: discount plans and pricing policies are interfaces with interchangeable implementations, so a new plan or policy is a new class rather than an edit to the checkout. The same reasoning applies to the delivery charging

scheme (R8). The explicit demand for the **Observer** pattern in R9 shaped the inventory module, where the stock count is decoupled from whoever reacts to a shortage.

Three further decisions are worth noting. First, the bank-side protocol (R4) is hidden behind a single *authorise* operation, a **Facade** that keeps the rest of the system independent of the banking details. Second, the code is organised in layers (domain entities, services, orchestration, user interface) so that the business logic does not depend on the way it is driven — a property that proved valuable when a second interface was later added. Third, all monetary output is produced with a fixed locale (period decimals, UTF-8), so the computed figures are identical on any machine that grades the project.

2.2 The Bill Computation Pipeline

`CashRegister.computeBill()` is the heart of the specified system. For the items in the cart it applies, in order:

1. the **quantity-based unit price** (R3), through `Item.unitPriceFor(qty)`;
2. the **category pricing policy** (R6), e.g. a 10% reduction on fruit and vegetables;
3. the customer's **discount plan** (R5) on the resulting subtotal — prime applies its 20% only when that subtotal reaches €50, platinum always applies 30%;
4. the **delivery fee** (R8) when a delivery was requested, reduced by the plan (R8b: prime pays half, platinum free).

The total is thus *items after plan* plus *plan-adjusted delivery*. Subscribing to a plan also charges its annual fee immediately (normal €0, prime €50, platinum €200), as required by §2.3.

Chapter 3

Architecture, Design Patterns and UML Models

This chapter describes how the specified system is structured, which design patterns realise its requirements, and how the result is documented with UML models.

3.1 Package Architecture

The code is organised in layers, each in its own package, so that dependencies flow in a single direction: from the user interface, to the orchestration core, to the services, and finally to the domain entities.

- `model` and `users` hold the domain entities (`Item`, `Category`, `Cart`, `CartEntry`, `BankCard`, `Customer`, `Receipt`) and the user roles (`Manager`, `Cashier`).
- `discount` and `pricing` contain the discount-plan and pricing-policy strategies (with the discount-plan factory).
- `inventory` implements the live stock with its Observer-based alerts; `payment` models the POS and the bank authorisation system; `delivery` computes delivery fees and manages the time slots.
- `register` contains the orchestration: `CashRegister` runs a checkout session and `Supermarket` is the system core.
- `cli` provides the command-line user interface.

Because the business logic lives entirely in the core and the services, the `cli` package is only a *driver* of the `Supermarket` API. This separation of concerns keeps the domain testable in isolation and, as shown in Part II, allows a second user interface to be added without touching the domain.

3.2 Design Patterns

The specified system uses the four patterns of Table 3.1; each is introduced to solve a specific requirement.

Pattern	Where	Problem it solves
Strategy	`DiscountPlan` (Normal/Prime/Platinum)	Swap the discount behaviour per customer without conditionals; new plans need no change to the checkout (R5b).
Strategy	`PricingPolicy` per `Category`	Per-category pricing that can be changed or added at runtime (R6b).
Strategy	`DeliveryCalculator`	The delivery charging scheme can be replaced without touching the bill logic (R8).
Factory	`DiscountPlanFactory`	Creates a plan from its name (used by `subscribeToPlan`), isolating the system from the concrete plan classes.
Observer	`Inventory` → `StockObserver`	Decouples stock bookkeeping from whoever reacts to a shortage; new listeners subscribe freely (R9).
Facade	`Transaction AuthorisationSystem`	Hides the banking authorisation protocol behind a single `authorise(card, amount)` operation (R4).

Table 3.1: Design patterns used in the specified system.

The three **Strategy** uses share the same intent — making a family of behaviours interchangeable — but target different axes of variation: the customer's plan, the category's pricing and the delivery charging. The **Factory** complements the discount Strategy by turning a plan name (a string typed at the CLUI) into the right object. The **Observer** is the one pattern explicitly mandated by the instructions: when a perishable item drops to or below its threshold, `Inventory` notifies every registered `StockObserver` (a manager notifier and a supplier notifier), with no knowledge of what they do. The **Facade** concentrates the bank dialogue — PIN verification on the card chip and balance authorisation — behind one method, so the checkout never manipulates banking internals directly.

3.3 Domain Core and Orchestration

Two classes coordinate the rest. `Supermarket` is the *system core*: it owns the catalogue, the categories, the registered users, the inventory, the bank authorisation system and the cash register, and it exposes the operations invoked by the CLUI (registering users, adding items, setting discounts, requesting delivery, opening a checkout). `CashRegister` is the *checkout orchestrator*: it holds the current customer and cart, computes the bill across the cart, the pricing policies, the plan and the delivery, and drives the POS payment. On a successful payment it decrements the inventory — possibly triggering the R9 alert — records the revenue and prints the receipt. This mediator-like arrangement keeps each collaborating service simple while a single object owns the checkout workflow. The remainder of this chapter presents the UML models of the specified system. Because the full class diagram is large, it is shown here as focused per-package views, each legible on its own; the complete class diagram is provided in Appendix B. The extension classes are shown separately in Part II.

3.4 Class Diagram (per package)

The class model follows the package layering introduced earlier in this chapter. Figure 3.1 shows the domain entities: a `Customer` (a `User`) holds a discount plan and a bank card, an `Item` belongs to a `Category` carrying its pricing policy, and a `Cart` aggregates `CartEntry` lines. Figure 3.2 shows the two Strategy families — discount plans (with their factory) and pricing policies. The Observer and Facade structures appear in Figure 3.3: the `Inventory` notifying its `StockObserver`s on the left, and the POS and the bank authorisation system on the right. Figure 3.4 gathers the delivery and smart-logistics classes, and Figure 3.5 shows how `Supermarket` and `CashRegister` wire the services together and how the CLUI drives the core.

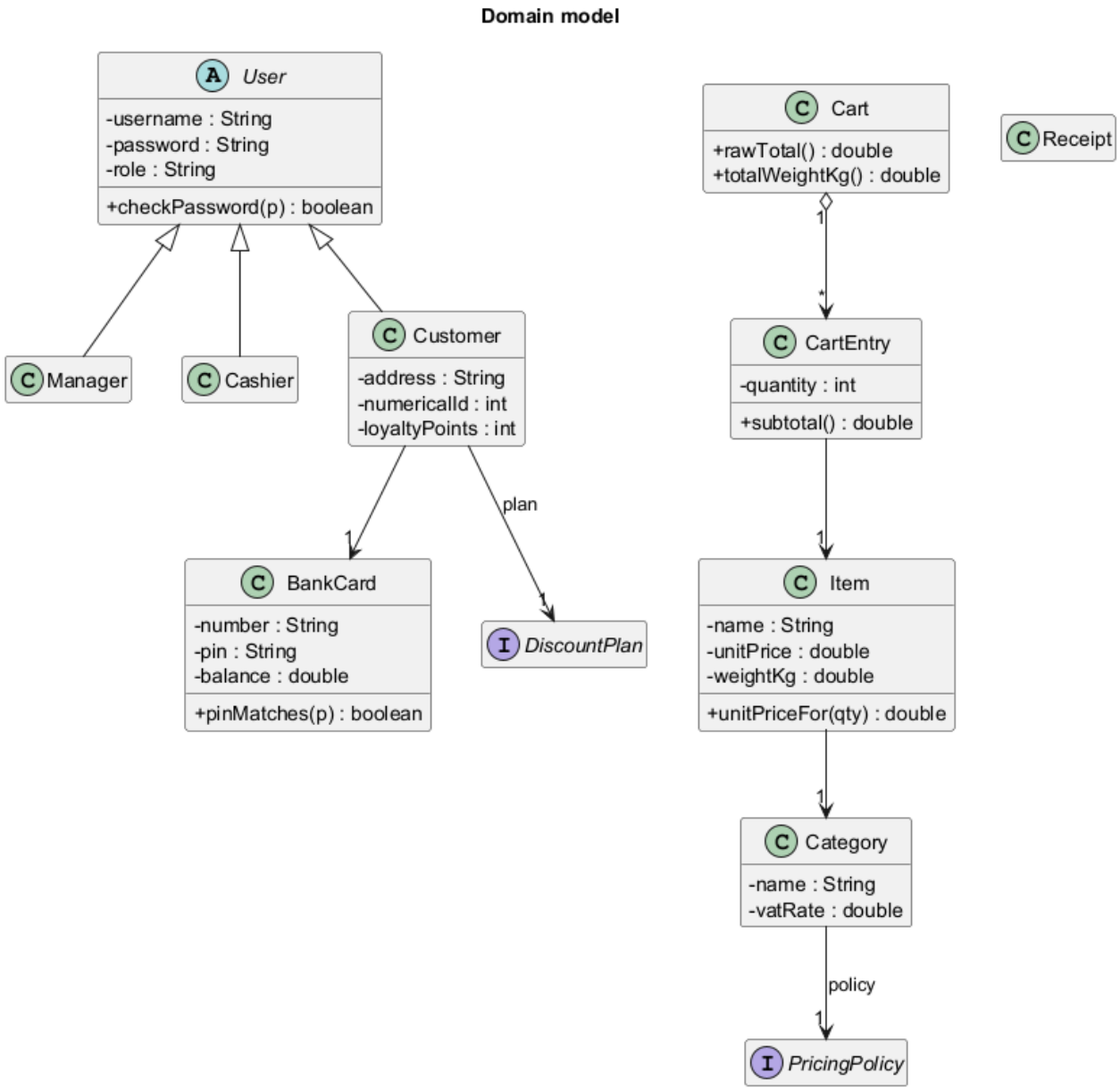


Figure 3.1: Domain model: entities, the user hierarchy and their relationships.

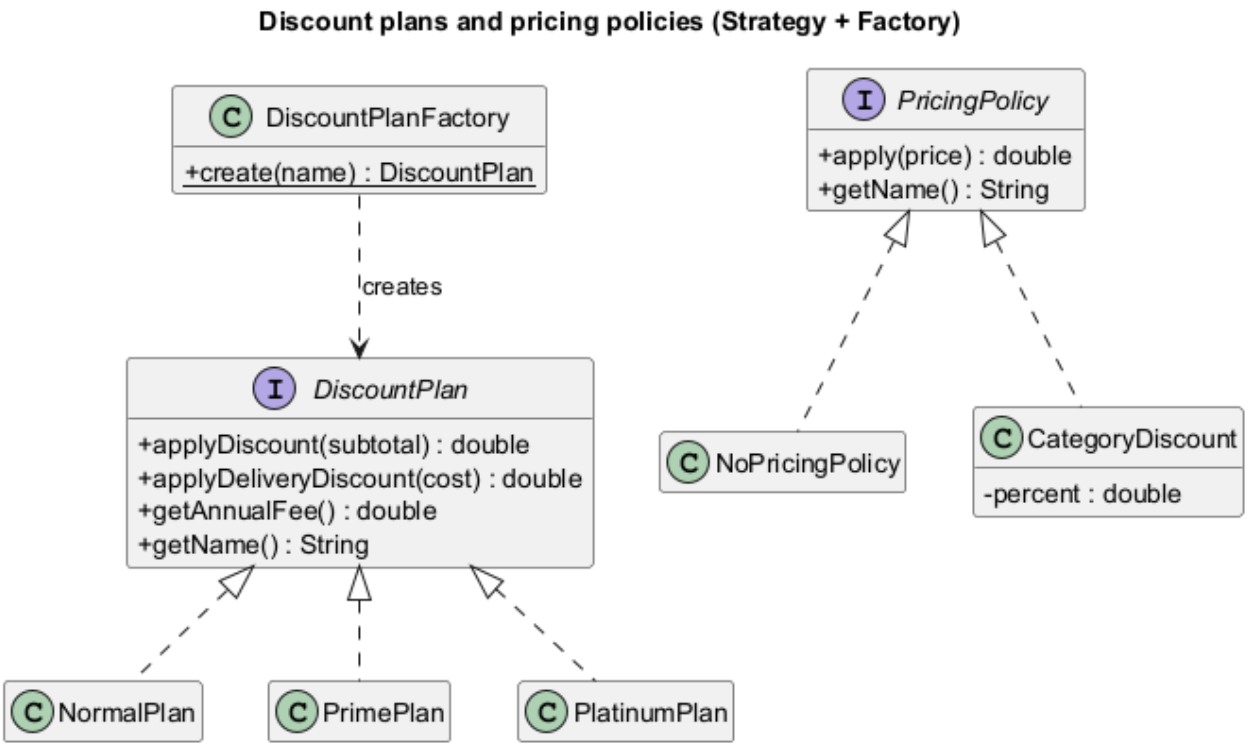
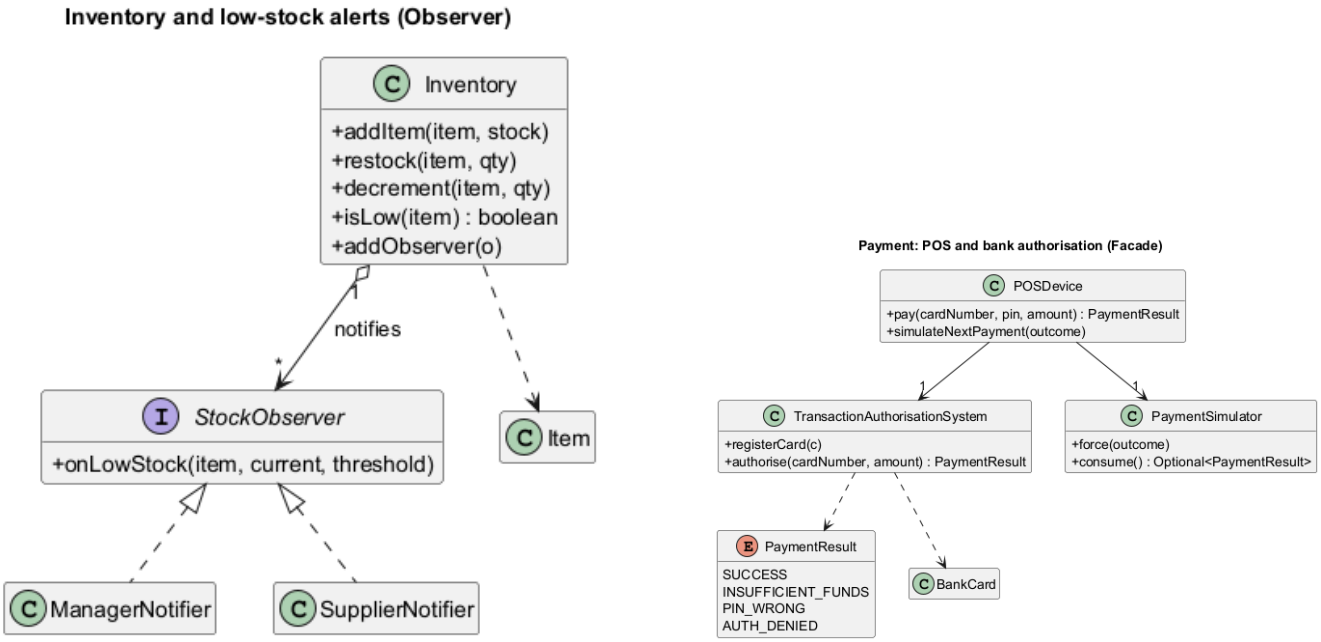


Figure 3.2: Discount plans and pricing policies (Strategy + Factory).



(a) Inventory and alerts (Observer).

(b) Payment (Facade).

Figure 3.3: Inventory/Observer (left) and Payment/Facade (right).

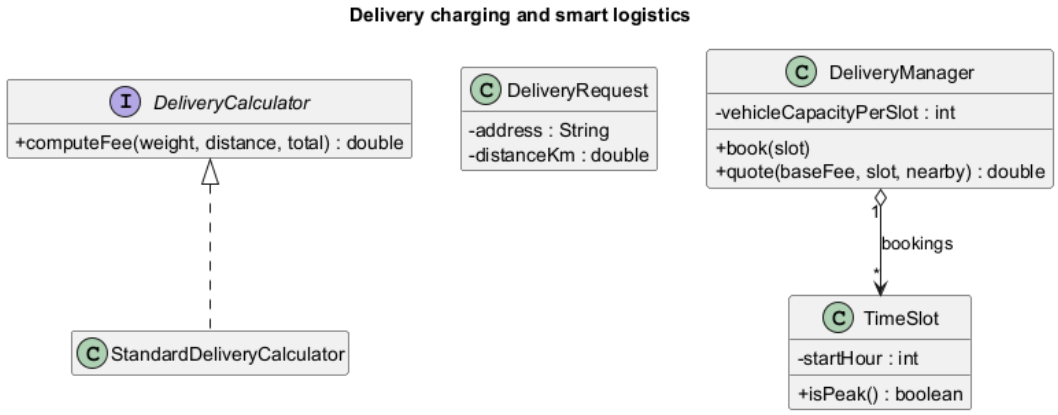


Figure 3.4: Delivery charging (Strategy) and smart-logistics classes.

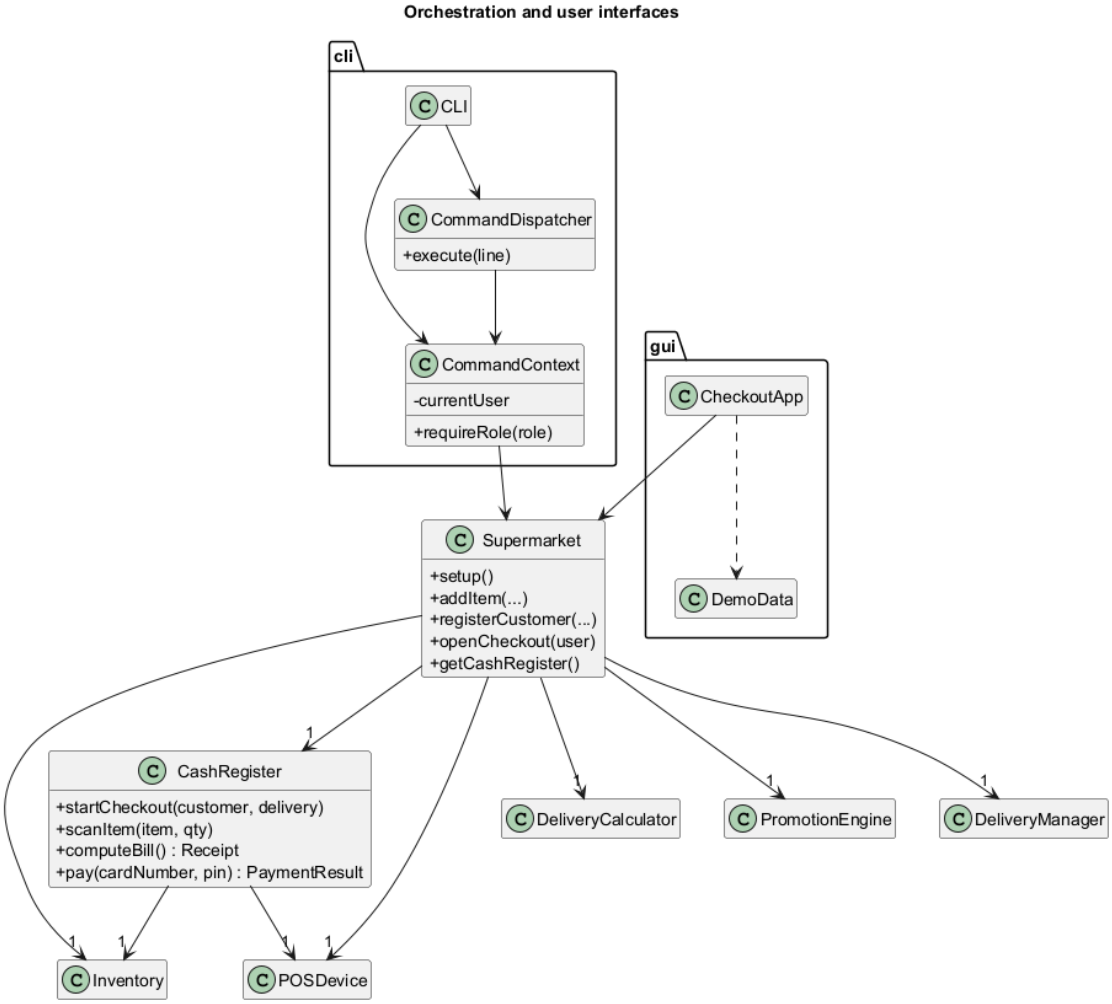


Figure 3.5: Orchestration core (`Supermarket`, `CashRegister`) and the two user interfaces.

3.5 Use-Case Diagram

Figure 3.6 summarises the interactions of the three human actors and the bank. The *Manager* configures the system (setup, registering users, adding items, setting category discounts, restocking, inspecting inventory and revenue);

the *Cashier* runs the checkout (start, scan, compute bill, pay, simulate a payment outcome); and the *Customer* subscribes to a plan and requests home delivery. Two relationships are worth noting: *Pay includes* the bank's *Authorise transaction*, and it *extends* into *Notify low stock* when a sale brings a perishable item below its threshold (R9).

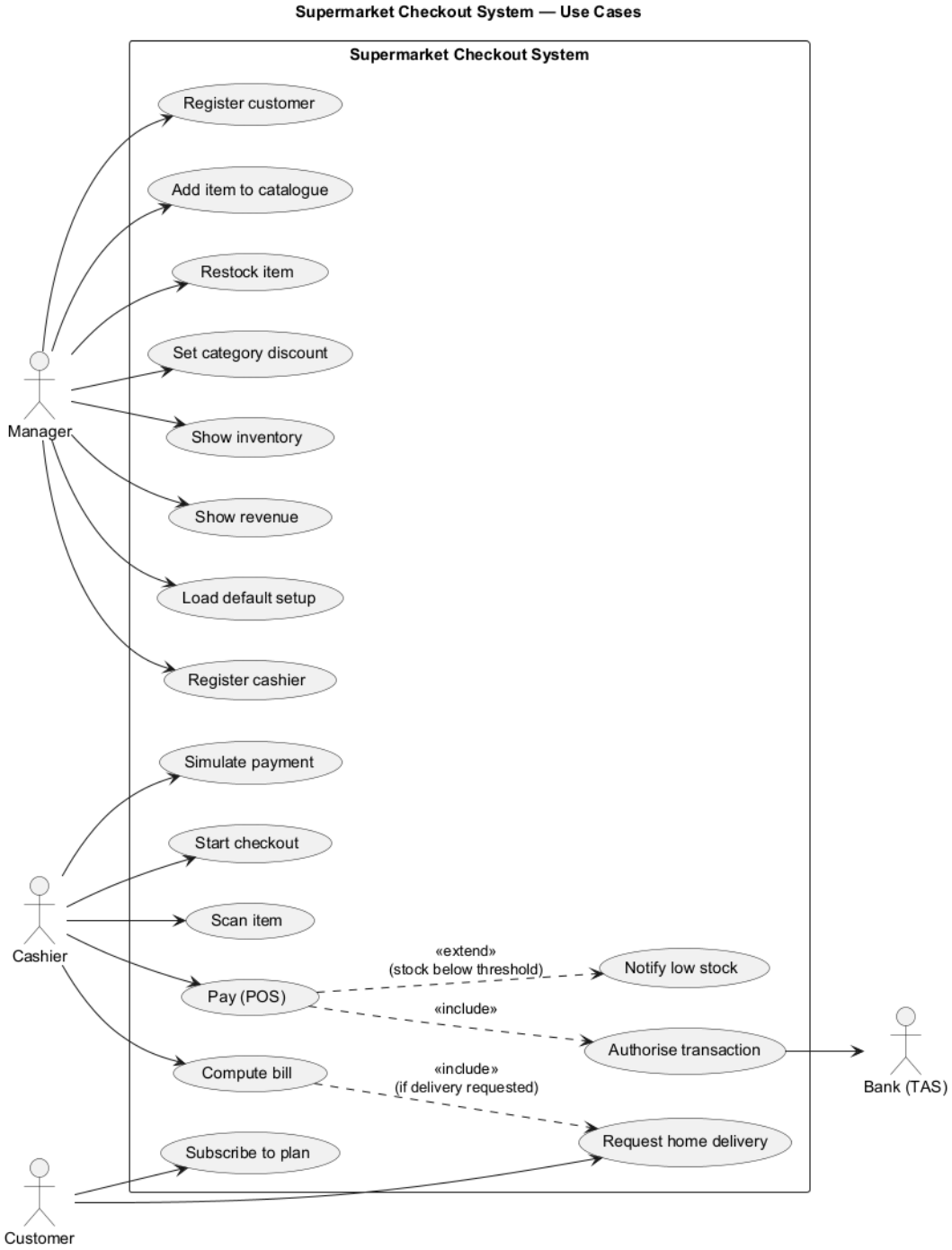


Figure 3.6: Use cases for the Manager, Cashier, Customer and the Bank (TAS).

3.6 Sequence Diagrams

The checkout is split into two diagrams for clarity. Figure 3.7 shows the bill computation: the cashier opens the session, scans items into the cart and asks for the bill, which the cash register computes across the cart, the discount

plan and the delivery calculator. Figure 3.8 shows the payment: the POS either honours a forced outcome (used for reproducible tests) or runs the real flow through the bank, and on success the inventory is decremented and the receipt is returned. Figure 3.9 then zooms into the Observer mechanism of R9: when `decrement` brings a perishable item to or below its threshold, `Inventory` notifies each observer in turn.

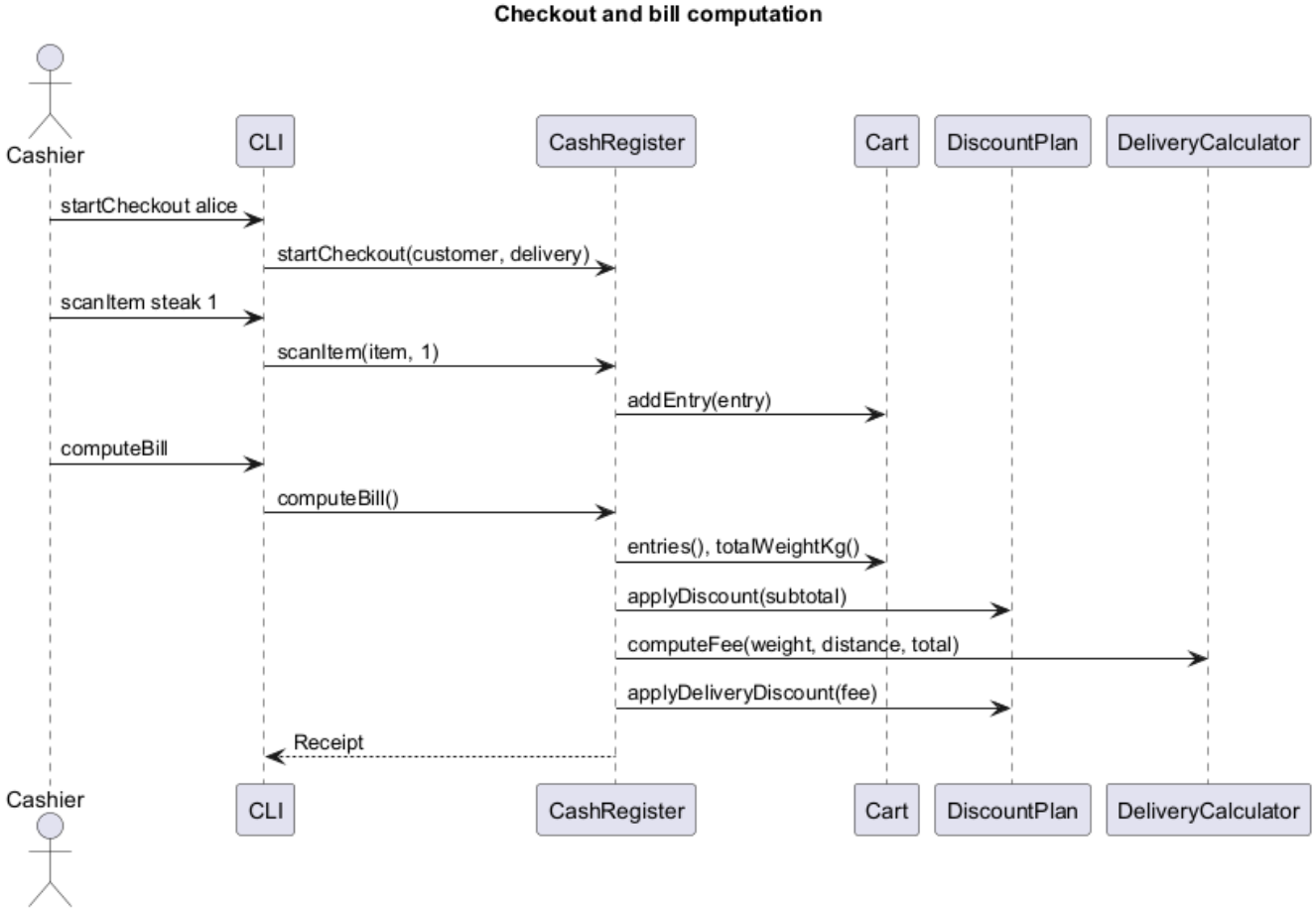


Figure 3.7: Opening a checkout, scanning items and computing the bill.

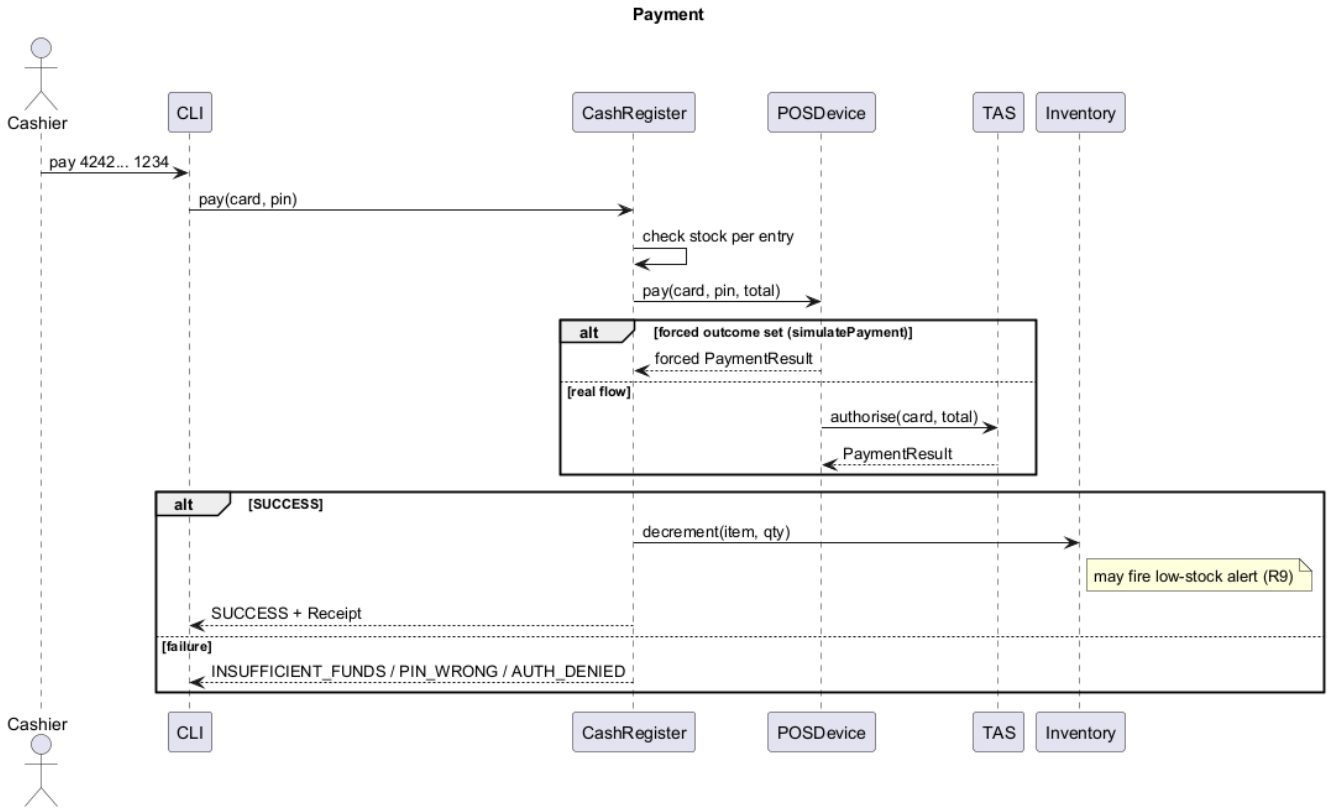


Figure 3.8: Payment flow, including the forced/real payment branch and the post-success inventory decrement.

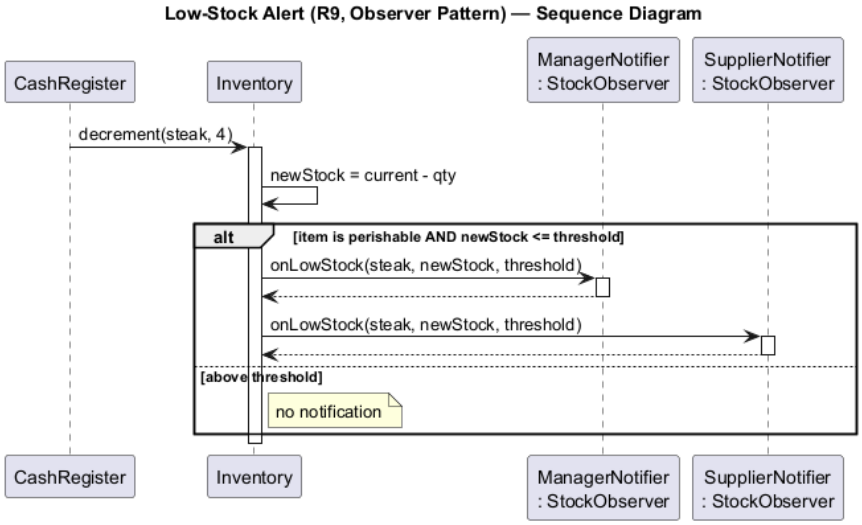


Figure 3.9: Low-stock alert via the Observer pattern (R9).

Chapter 4

Testing

Testing is a mandatory part of the project and was treated as a first-class concern: the system is validated both by unit tests, written one class at a time, and by end-to-end scenarios executed through the CLUI.

4.1 JUnit Test Strategy

The suite follows the instruction to provide tests for the most significant methods of each class, excluding trivial getters and setters. The project statement supplies no JUnit tests of its own — only the format of the command-line test scenarios — so every test class in the suite was written specifically for this project. It is organised as one test class per unit, mirroring the source packages: bill computation (`BillComputationTest`), the payment flow and the bank authorisation (`PaymentFlowTest`, `TransactionAuthorisationSystemTest`), the inventory and its Observer (`InventoryObserverTest`), the delivery charging and logistics (`DeliveryCalculatorTest`, `DeliveryManagerTest`, `TimeSlotTest`), the discount plans and pricing policies (`DiscountPlanTest`, `PricingPolicyTest`), and the system core and CLUI (`SupermarketTest`, `CliDispatcherTest`). The specified system is covered by 61 tests in the `v1.0` version (76 in the final version, which adds the extension tests). The CLUI dispatcher writes to an injectable stream, so the entire command interface is tested without any real console.

4.2 Test Scenarios

A scenario file is a list of CLUI commands run with `runTest`; comment lines beginning with `#` are ignored. The mandatory `testScenario1.txt` reproduces a complete purchase; two further scenarios exercise the remaining specified features. Crucially, the JUnit `CliDispatcherTest` executes the very same scenario files, so the figures quoted here are checked automatically.

Scenario	What it tests
`testScenario1.txt`	Core happy path: a −10% category discount, a prime subscription (€50 fee), a mixed-category checkout, the prime-halved delivery fee (€7.50), a failed (`INSUFFICIENT_FUNDS`) then successful payment, and a low-stock alert when steak reaches 0. Final revenue €120.74.
`testScenario2.txt`	Platinum plan (−30%, free delivery, €200 fee), a dairy discount, the `PIN_WRONG` and `AUTH_DENIED` payment paths, a low-stock alert cleared by `restock`, and a delivery refusal above 50 kg. Final revenue €254.236.
`testScenario3.txt`	Quantity-based bulk pricing (R3) and the smart logistics (R10): slot booking with anti-overbooking and dynamic pricing (peak €22.50, eco €12.00). Final revenue €15.00.

Table 4.1: Test scenarios for the specified system. The extensions are exercised by a fourth scenario described in Part II.

4.3 How to Reproduce the Tests

The whole suite runs with a single command, and each scenario can be replayed inside the CLUI:

```
mvn test                  # compile and run every JUnit test
mvn exec:java -Dexec.mainClass="supermarket.cli.CLI"
> runTest testScenario1.txt
```

Because the scenario files are read from the project root, the automated tests and the interactive `runTest` execute the same command sequences, which makes every reported result reproducible.

Part II

Extensions Beyond the Specification

Chapter 5

The Optional Extensions

Part I — the *specified system* — corresponds exactly to what the project statement asked for: requirements R1–R10, the command-line interface and the mandatory tests, and nothing more. **This part goes beyond that brief.** None of the features described below were requested by the assignment; they were added on my own initiative, to push the design further and to show that it is genuinely open to extension. They are built on the specification baseline (tagged `v1.0`) and are deliberately *additive*: every extension defaults to a no-op, so the required behaviour — and the figures of Part I — are unchanged when no promotion, tax or loyalty rule is configured. The two runnable versions (specified-only and final) are described in the README and Appendix A. Figure 5.1 shows the classes introduced here and how they plug into `CashRegister`.

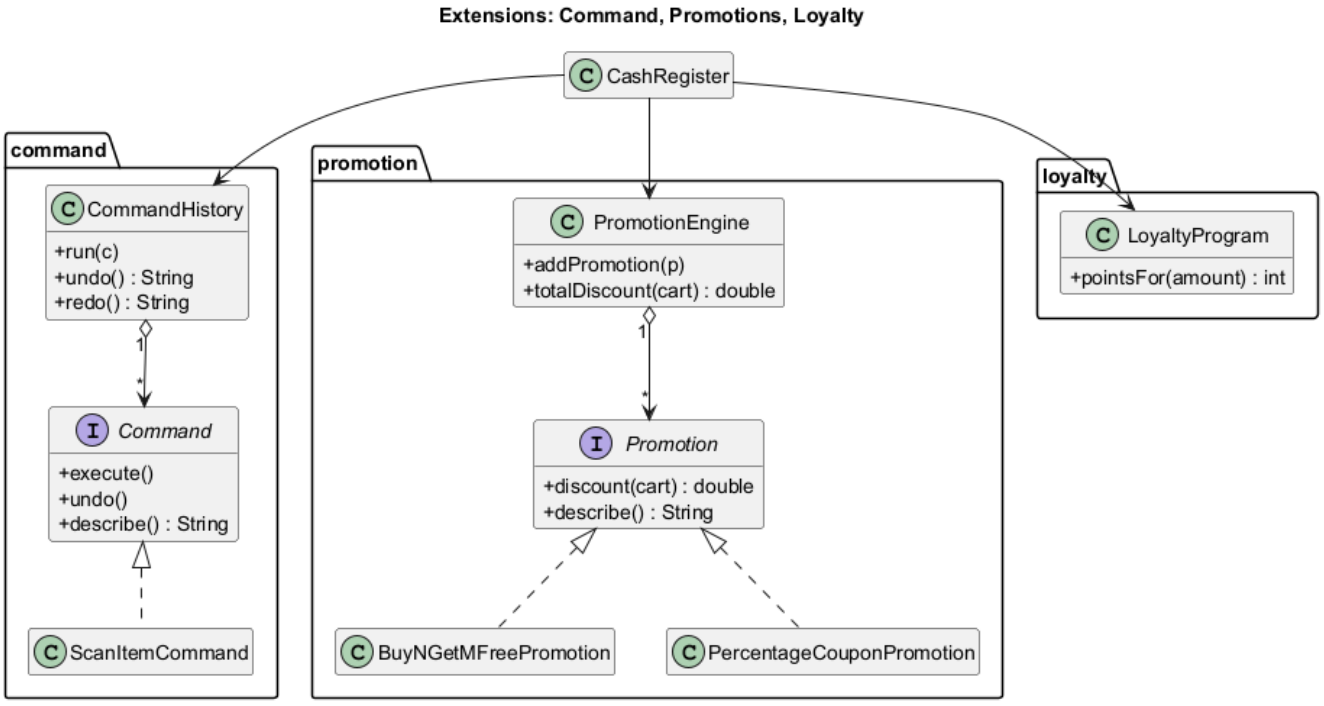


Figure 5.1: Class diagram of the extensions: Command (undo/redo), Promotions (Strategy) and Loyalty, plugged into `CashRegister`.

5.1 Undo/Redo (Command Pattern)

A cashier may scan a wrong item or quantity. To make such an action reversible, each scan is turned into a **Command**: `scanItem` no longer adds a cart line directly but runs a `ScanItemCommand` through a `CommandHistory`. The history is the *invoker* of the pattern — it executes the command and keeps an undo stack and a redo stack — while each `Command` knows how to `execute`, `undo` and `describe` itself. The CLUI exposes the new `undo` and `redo` commands, so a mis-scan is corrected with a single keystroke instead of restarting the checkout. The pattern keeps the cash register oblivious to *how* an action is reversed.

5.2 Promotions Engine (Strategy)

Store promotions are modelled as another **Strategy**. The `Promotion` interface returns the discount it grants for a given cart, and two concrete promotions are provided: `BuyNGetMFreePromotion` (for every $N + M$ units of an item, M are free) and `PercentageCouponPromotion` (a percentage off the cart). A `PromotionEngine` aggregates the active promotions and returns their combined discount; new kinds of promotion are added by implementing the interface, with no change to the engine or the checkout. The manager configures them with the `addBogoPromotion` and `addCoupon` commands.

5.3 Per-Category VAT

Each `Category` gains an optional VAT rate (zero by default). During bill computation the tax is accumulated on the net, post-category-policy price of each line and added to the total; it is shown on the receipt only when non-zero. The manager sets it with `setCategoryTax`. Modelling VAT at the category level mirrors the design of the category pricing policy and keeps the rule close to the entity it concerns.

5.4 Loyalty Points

A small `LoyaltyProgram` service awards one point per euro spent. On a successful payment the cash register asks the program how many points the total earns and credits them to the customer, who can later check the balance with the `showPoints` command. Keeping the earning rule in a single service makes it trivial to change the rate or introduce point redemption later.

5.5 The Extended Bill and a Demonstration

With the extensions enabled, the bill pipeline gains two stages around the plan discount: promotions are subtracted from the items subtotal (after the category policy, before the plan, capped so it never goes negative), and VAT is added at the end. The full order is therefore: quantity price, category policy, *promotions*, discount plan, *VAT*, delivery.

The scenario `testScenario4.txt` exercises all four extensions end to end. A mis-scan is corrected with `undo`/`redo`; a "buy one get one free" offer on soda and a 5% coupon are applied; the grocery category carries a 10% VAT. For a cart of four sodas and three units of rice, the raw total of €18.00 is reduced by €6.90 of promotions to €11.10, to which €1.80 of VAT is added, giving a total of €12.90 and 12 loyalty points. These extensions are covered by dedicated unit tests (`CommandUndoRedoTest`, `PromotionEngineTest`, `LoyaltyProgramTest`, `BillExtrasTest`) and by `GuiDomainFlowTest`; the full project totals 76 JUnit tests.

5.6 JavaFX Graphical Interface

The clearest evidence of the architecture's quality is that a second user interface could be added *without changing the domain at all*. The `gui` package provides a JavaFX desktop application (`CheckoutApp`) that is purely a presentation layer over the same `Supermarket` core used by the CLUI. It shows the catalogue, the live cart and the bill breakdown, offers buttons to scan, `undo`/`redo` and pay (with a selectable simulated outcome), and displays the revenue and the customer's loyalty points. The inventory table highlights low-stock rows in real time through a *new* `StockObserver` registered on the existing `Inventory` — a direct, visual demonstration of the Observer pattern of R9. The interface is launched with `mvn javafx:run`. Figure 5.2 shows the running application after a sale: the out-of-stock *steak* row is highlighted in red by the `StockObserver`, and the events panel records the scans, the approved payment and the loyalty points earned.

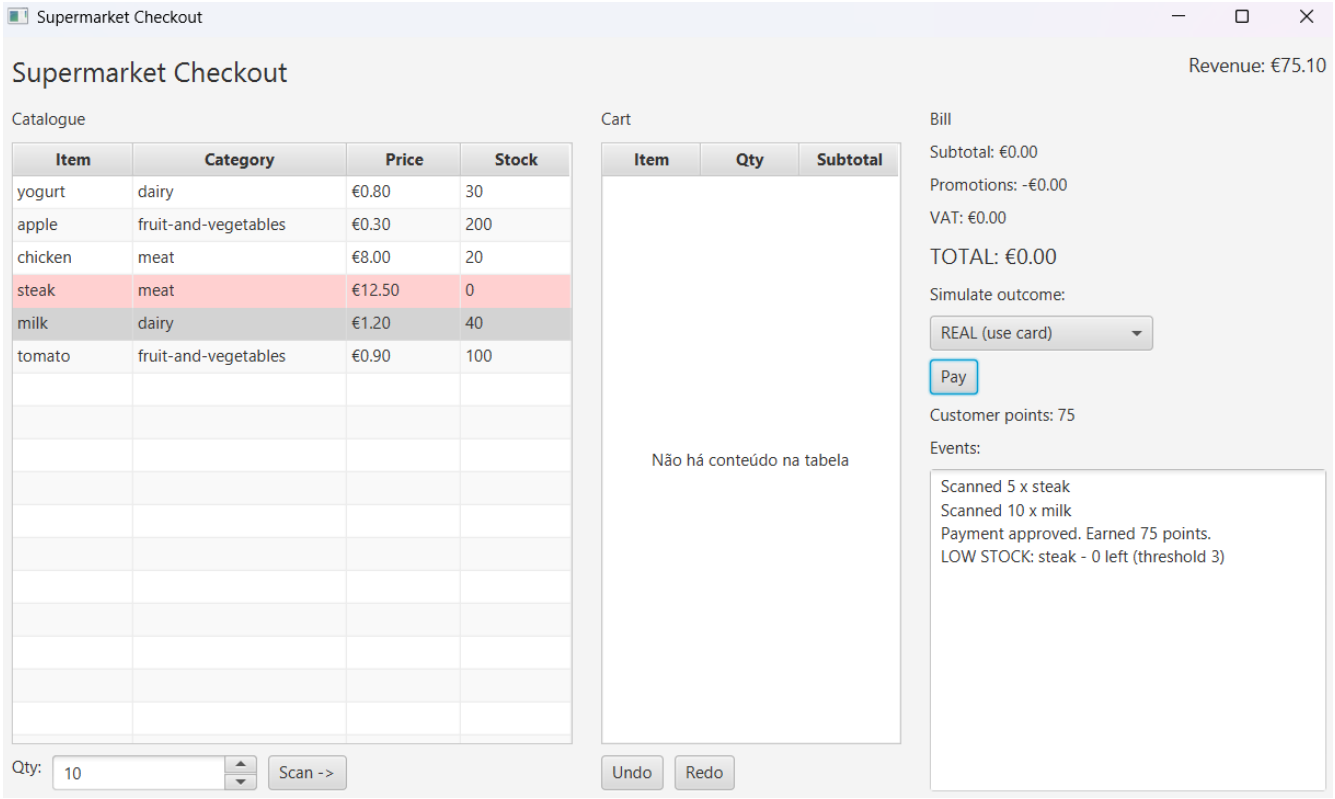


Figure 5.2: The JavaFX desktop interface — a second presentation layer over the same `Supermarket` core used by the CLUI, demonstrating the Observer low-stock highlight (R9) and the loyalty points.

Chapter 6

Design Analysis and Conclusion

6.1 Advantages of the Solution

The implementation satisfies every requirement of the specification, but its main quality lies in how well it accommodates change. Three properties stand out. First, the design is *open to extension and closed to modification*: thanks to the Strategy and Factory patterns, a new discount plan, pricing policy, delivery scheme or promotion is introduced as a new class, with no edit to the checkout logic. The four extensions of Part II were added in exactly this way, without altering the specified behaviour. Second, the strict *separation of concerns* — domain entities, services, orchestration and user interface in distinct layers — keeps the business logic independent of the way it is driven; the clearest evidence is that a complete JavaFX interface could be built over the same core without changing a single domain class. Third, the system produces *reproducible output*: a fixed locale makes every monetary figure deterministic, and the CLUI scenario files double as JUnit fixtures, so each result quoted in this report is checked automatically.

6.2 Workload Split

This is an individual project: every design decision, line of code, UML diagram and test was produced by the single author, as summarised in Table 6.1.

Module / Task	**Design**	**Code**	**UML**	**Tests**
Domain model and users	✓	✓	✓	✓
Discount plans and pricing policies	✓	✓	✓	✓
Inventory and stock alerts (Observer)	✓	✓	✓	✓
Payment (POS and bank authorisation)	✓	✓	✓	✓
Delivery and smart logistics	✓	✓	✓	✓
Orchestration (`Supermarket`, `CashRegister`)	✓	✓	✓	✓
Command-line interface	✓	✓	✓	✓
Extensions (Command, promotions, VAT, loyalty)	✓	✓	✓	✓
JavaFX graphical interface	✓	✓	✓	✓

Table 6.1: Workload split (Enzo Jardim Vendramin).

While developing the project I also made occasional use of artificial-intelligence assistance, which I regard as part of the learning process: before this course I had never written a line of Java, nor used object-oriented programming at all. The lectures gave me the foundations, and this project pushed me well beyond them. I genuinely enjoyed the content, the challenge of solving each problem, and the opportunity to apply it to a concrete, everyday setting such as a supermarket checkout.

6.3 Conclusion

The project delivers a complete supermarket checkout system that meets the full specification — the bought-items model, quantity-based and category pricing, extensible discount plans, bank-card payment, weight-and-distance delivery, the Observer-based inventory and the smart-logistics slots — all driven through a command-line interface and validated by an automated test suite (Part I). On top of this verified baseline, Part II shows that the same design readily supports features that were never asked for: reversible actions, a promotions engine, per-category VAT, a loyalty programme and an entire second user interface. The exercise therefore meets its twofold objective: it implements what the specification requires, and it does so with a pattern-driven, layered design whose value is demonstrated — rather than merely claimed — by how easily it grew.

Appendix A

How to Build and Run

The project is built with Maven and requires a Java Development Kit version 17 or later (it was developed on JDK 21) together with Maven 3.9 or later. The repository offers two runnable versions of the same code base: the *specification version*, tagged `v1.0`, which contains exactly what the instructions require, and the *final version* on the `master` branch, which also includes the extensions of Part II and the graphical interface. Selecting a version is simply a matter of checking out the corresponding tag or branch before building, as shown in Listing A.1.

```
# --- choose the version to run ------------------------------------------
git checkout v1.0      # specification-only version (R1-R10, CLUI, tests)
git checkout master    # final version (adds the optional extensions + GUI)

# --- build, test and run ------------------------------------------------
mvn test               # compile and run the whole JUnit suite
mvn exec:java          # launch the command-line interface (CLUI)
mvn javafx:run         # launch the JavaFX desktop GUI (final version only)
```

Listing A.1: Build, test and run commands.

Inside the CLUI, a complete end-to-end scenario is replayed with, for example, `runTest testScenario1.txt`. The same scenario files are executed automatically by `CliDispatcherTest`, so the interactive session and the test suite exercise identical command sequences.

Appendix B

Complete Class Diagram

For completeness, the full class diagram of the whole system (specified core and extensions) is reproduced below in landscape. The per-package views in Section 3.4 are the legible decomposition of this diagram.

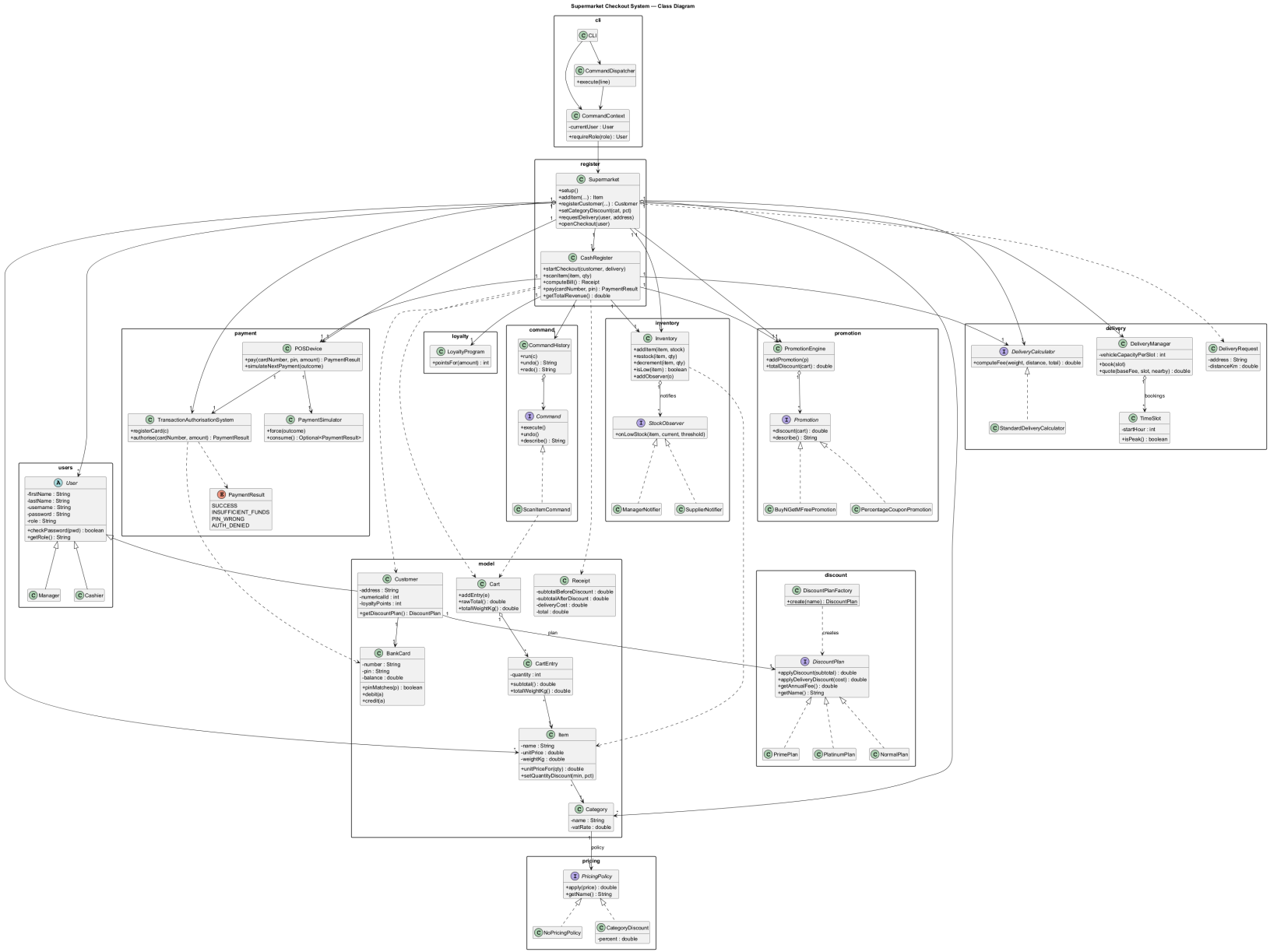


Figure B.1: Complete class diagram (all packages).

Appendix C

Selected Code Listings

The following excerpts illustrate the central design choice of the specified system: a discount plan is a **Strategy** (Listing C.1) with interchangeable concrete implementations (Listing C.2), instantiated from its name by a **Factory** (Listing C.3). Introducing a new plan therefore means writing one new class and adding a single line to the factory, with no change to the checkout logic.

```java
public interface DiscountPlan {
    double applyDiscount(double subtotal);
    double applyDeliveryDiscount(double deliveryCost);
    String getName();

    /** Annual fee charged immediately when a customer subscribes (spec 2.3). */
    double getAnnualFee();
}
```

Listing C.1: The `DiscountPlan` strategy interface.

```java
public class PrimePlan implements DiscountPlan {
    @Override
    public double applyDiscount(double subtotal) {
        return subtotal >= 50.0 ? subtotal * 0.80 : subtotal;
    }

    @Override
    public double applyDeliveryDiscount(double deliveryCost) {
        return deliveryCost * 0.50;
    }

    @Override
    public String getName() { return "prime"; }

    @Override
    public double getAnnualFee() { return 50.0; }
}
```

Listing C.2: A concrete strategy: the `PrimePlan`.

```java
public class DiscountPlanFactory {
    public static DiscountPlan create(String name) {
        return switch (name.toLowerCase()) {
            case "normal"   -> new NormalPlan();
            case "prime"    -> new PrimePlan();
            case "platinum" -> new PlatinumPlan();
            default -> throw new IllegalArgumentException("Unknown plan: " + name);
        };
    }
}
```

Listing C.3: The `DiscountPlanFactory`: a plan name becomes the right object.